МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра интеллектуально информационных технологий

Отчет по лабораторной работе №4

Специальность ПО-12

Выполнил:

Середич К. Н.

Студент группы ПО-12

Проверила:

Кулик А. Д.

Преподаватель-стажер

кафедры ИИТ

Брест 2026

Цель работы: научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API или GraphQL API

Общее задание: используя Github API, реализовать предложенное задание на языке Python. Выполнить визуализацию результатов, с использованием графика или отчета. Можно использовать как REST API (рекомендуется), так и GraphQL

1. Автоматический анализ трендов популярных репозиториев

Условие:

Напишите Python-скрипт, который анализирует тренды популярных GitHub-репозиториев за определённый период (например, за последние 7 или 30 дней).

1. Запрашивает у пользователя:
   Язык программирования (например, Python, JavaScript, Go)
   Период анализа (за неделю, за месяц)
   (Опционально) Минимальное количество звезд, чтобы фильтровать менее популярные проекты
2. Использует GitHub API для получения самых быстрорастущих репозиториев по звёздам (stars).
3. Для каждого найденного репозитория собирает данные:
   Название
   Автор
   Количество звёзд за период
   Общее количество звёзд
   Количество форков
   Основной язык
   Описание
   Ссылка на репозиторий
4. Определяет самые динамично растущие проекты (по количеству новых звёзд за период).
5. Визуализирует динамику роста популярных проектов (matplotlib, seaborn).

Введите язык программирования: Python

Выберите период (7 / 30 дней): 7

Минимальное количество звёзд (по желанию): 500

Анализируем популярные репозитории на Python за последние 7 дней...

ТОП-5 самых быстрорастущих проектов:

1. **repo1** (+3,500 ☆) - новый фреймворк для ML
2. **repo2** (+2,800 ☆) - библиотека для работы с API
3. **repo3** (+2,500 ☆) - CLI-инструмент для DevOps
4. **repo4** (+2,000 ☆) - новый веб-фреймворк
5. **repo5** (+1,700 ☆) - утилита для работы с графами

Графики роста сохранены в "trending_python.png"

Спецификация ввода-вывода программы:

Входные данные:

Пользователь вводит:

язык программирования (строка);

период анализа (целое число от 1 до 30);

минимальное количество звёзд (целое число или пустое значение).

Пример:

Введите язык программирования: Python

Введите период анализа (от 1 до 30 дней): 7

Минимальное количество звёзд (по желанию): 500

Выходные данные:

Программа выводит:

список ТОП-5 популярных репозиториев;

информацию о каждом репозитории;

путь к сохранённому графику.

Пример:

ТОП-5 самых быстрорастущих проектов:

1. repo1 (3500 ☆) - описание

...

График сохранён в '...\trending_python.png'

Код программы:

```python
"""
Анализ трендов GitHub-репозиториев.

Скрипт получает популярные репозитории по заданному языку
и строит график их популярности.
"""

import os
import datetime
import requests
import matplotlib.pyplot as plt


GITHUB_API_URL = "https://api.github.com/search/repositories"


def get_user_input():
    """Получение параметров от пользователя."""
    language = input("Введите язык программирования: ").strip()

    period_input = input("Введите период анализа (от 1 до 30 дней): ").strip()

    if not period_input.isdigit():
        raise ValueError("Период должен быть числом")

    period = int(period_input)

    if period < 1 or period > 30:
        raise ValueError("Период должен быть от 1 до 30 дней")

    min_stars_input = input("Минимальное количество звёзд (по желанию): ").strip()

    min_stars = int(min_stars_input) if min_stars_input else 0

    return language, period, min_stars


def build_query(language, period, min_stars):
    """Формирование поискового запроса."""
    date_from = datetime.datetime.now() - datetime.timedelta(days=period)
    date_str = date_from.strftime("%Y-%m-%d")

    query = f"language:{language} created:>{date_str} stars:>={min_stars}"
    return query


def fetch_repositories(query):
    """Получение репозиториев через GitHub API."""
    params = {
        "q": query,
        "sort": "stars",
        "order": "desc",
        "per_page": 10,
    }

    response = requests.get(GITHUB_API_URL, params=params, timeout=10)
    response.raise_for_status()
```

```python
    return response.json().get("items", [])


def extract_repo_data(repos):
    """Извлечение нужных данных."""
    result = []

    for repo in repos:
        repo_info = {
            "name": repo["name"],
            "author": repo["owner"]["login"],
            "stars": repo["stargazers_count"],
            "forks": repo["forks_count"],
            "language": repo["language"],
            "description": repo["description"],
            "url": repo["html_url"],
        }
        result.append(repo_info)

    return result


def print_top_repositories(repos):
    """Вывод ТОП-5 репозиториев."""
    print("\nТОП-5 самых быстрорастущих проектов:")

    for index, repo in enumerate(repos[:5], start=1):
        print(
            f"{index}. **{repo['name']}** "
            f"({repo['stars']} ★) - {repo['description']}"
        )


def plot_results(repos, language):
    """Построение и сохранение графика."""
    if not repos:
        print("Нет данных для построения графика")
        return

    names = [repo["name"] for repo in repos[:5]]
    stars = [repo["stars"] for repo in repos[:5]]

    plt.figure(figsize=(10, 6))
    bars = plt.bar(names, stars)

    plt.title(f"Топ репозиториев ({language})", fontsize=14)
    plt.xlabel("Репозитории")
    plt.ylabel("Количество звёзд")

    plt.xticks(rotation=30, ha="right")
    plt.grid(axis="y", linestyle="--", alpha=0.7)

    # Подписи значений на столбцах
    for bar_item in bars:
        height = bar_item.get_height()
        plt.text(
            bar_item.get_x() + bar_item.get_width() / 2,
            height,
            f"{int(height)}",
```

```python
            ha="center",
            va="bottom"
        )

    plt.tight_layout()

    filename = f"trending_{language.lower()}.png"

    script_dir = os.path.dirname(os.path.abspath(__file__))
    filepath = os.path.join(script_dir, filename)

    plt.savefig(filepath)
    plt.close()

    print(f"\nГрафик сохранён в '{filepath}'")


def main():
    """Основная функция."""
    try:
        language, period, min_stars = get_user_input()

        print(
            f"\nАнализируем популярные репозитории на {language} "
            f"за последние {period} дней..."
        )

        query = build_query(language, period, min_stars)
        repos = fetch_repositories(query)
        repo_data = extract_repo_data(repos)

        print_top_repositories(repo_data)
        plot_results(repo_data, language)

    except Exception as error:  # pylint: disable=broad-except
        print(f"Ошибка: {error}")


if __name__ == "__main__":
    main()
```

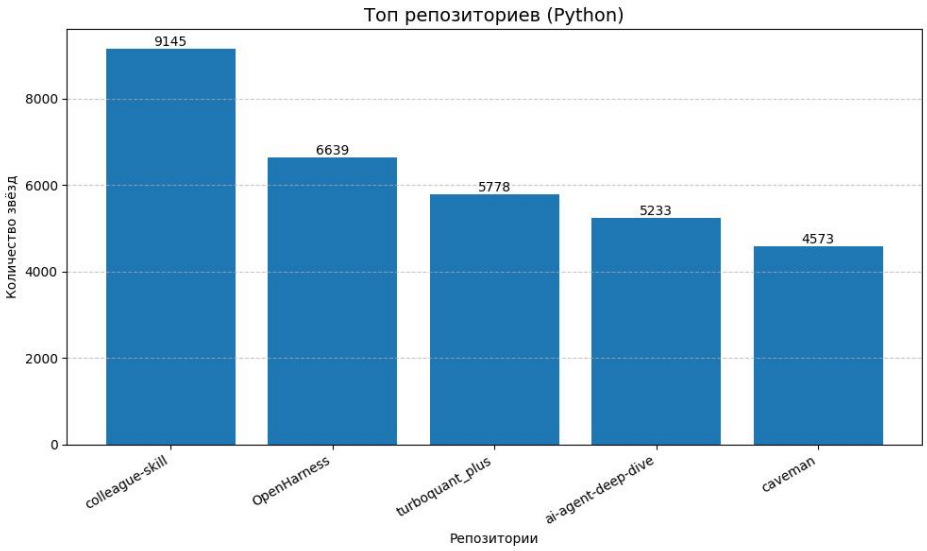
Результат работы программы:

```
Введите язык программирования: Python
Введите период анализа (от 1 до 30 дней): 23
Минимальное количество звёзд (по желанию): 4

Анализируем популярные репозитории на Python за последние 23 дней...

ТОП-5 самых быстрорастущих проектов:
1. **colleague-skill** (9145 ⭐) - 将冰冷的离别化为温暖的 Skill, 欢迎加入数字生命1.0! Transforming cold
 farewells into warm skills? It's giving rebirth era. Welcome to Digital Life 1.0. 🫂
2. **OpenHarness** (6639 ⭐) - "OpenHarness: Open Agent Harness"
3. **turboquant_plus** (5778 ⭐) - None
4. **ai-agent-deep-dive** (5233 ⭐) - AI Agent 源码深度研究报告
5. **caveman** (4573 ⭐) - 🪨why use many token when few token do trick — Claude Code skill that cuts 6
5% of tokens by talking like caveman

График сохранён в 'd:\spp_rep\spp_po12\reports\Seredich_K_N\Lab_4\src\trending_python.png'
```



Вывод: Мы научились работать с Github API, приобрели практические навыки написания программ для работы с REST API или GraphQL API.